

Sentinelleai — Audit Report

MagnetaERC20OFTAutoLiquidity.sol · 164c29e8-3dbb-4d31-b61f-184d7181be93 ·
2026-05-22 11 h 55 min 08 s

CAUTION 42 / 100 6 consolidated findings

MagnetaERC20OFTAutoLiquidity presents a combination of centralization and cross-chain security risks that collectively warrant a CAUTION verdict. The most significant concern (SC01:2026 – Access Control) is that a single EOA controls all privileged functions including the LayerZero OFT delegate role, tax exemptions, treasury redirection, and pause capability — a pattern directly analogous to high-profile bridge exploits. The fee-on-transfer design (SC02:2026 – Business Logic) is fundamentally incompatible with standard AMM DEX protocols, creating systematic LP drain risk if the token is listed without exemptions. No timelock or multisig is enforced anywhere in the contract. Several formalist findings (oracle staleness, ERC4626, upgrade authorization) are inapplicable to this contract's design and are rejected. The contract's internal tax logic and OFT integration are otherwise sound, but the centralization and DeFi-incompatibility risks must be addressed before production deployment.

Severity breakdown

1 CRITICAL 2 HIGH 1 MEDIUM 1 LOW 1 INFO

Consolidated findings

CRITICAL CVSS 9.1 SC01:2026 — Broken Access Control

Single-EOA Owner Controls All Privileged Functions Including LayerZero Delegate

The constructor passes `initialOwner` as both the Ownable owner and the LayerZero OFT delegate. This single EOA controls: treasury redirection (setTreasuryAddress), tax exemptions (setTaxExempt), pause/unpause of all transfers, and all LayerZero configuration (setPeer, setConfig, DVN thresholds). Compromise of this single key allows an attacker to redirect all tax revenue, grant themselves tax-free status, halt the token, and potentially forge cross-chain messages — directly replicating the attack surface exploited in Kelp DAO and similar bridge incidents. No timelock or multisig is enforced.

```
constructor (OFT + Ownable), setTaxExempt(), setTreasuryAddress(), pause(),
unpause()
```

→ Transfer ownership to a 3-of-5 multisig wallet immediately after deployment. Enforce a TimelockController with minDelay $\geq$ 24 hours on all privileged paths. Assign the LayerZero delegate role to a separate, hardware-key-secured address. Set requiredDVNCount $\geq$ 2 in LayerZero config at deployment.

Confirmed by: historian, economist, formalist

HIGH CVSS 7.5 SC01:2026 — Broken Access Control

LayerZero OFT Initialized with Single Delegate and No DVN Quorum Enforcement

The OFT is initialized with a single delegate address (`initialOwner`) and no on-chain enforcement of requiredDVNs.length $\geq$ 2 or optional DVN quorum. An attacker who obtains the delegate key can forge lzReceive payloads to mint arbitrary tokens cross-chain, exactly as demonstrated in the Kelp DAO exploit pattern. The contract provides no post-deployment guard against leaving this configuration in a vulnerable state.

```
OFT(name_, symbol_, _lzEndpoint, initialOwner) in constructor
```

→ After deployment, immediately call setConfig to enforce requiredDVNs.length $\geq$ 2 and add an executor allowlist. Never leave the delegate role as a plain EOA. Document and enforce this as a deployment checklist requirement.

Confirmed by: historian

HIGH CVSS 7.2 SC02:2026 – Business Logic / Economic Design Flaw

Fee-on-Transfer Mechanism Incompatible with Standard AMM DEX Protocols

The 2% transfer tax implemented in `\_update` causes the recipient to receive `amount - tax` while the sender's balance is debited by `amount`. Standard AMM DEXs (Uniswap V2/V3) assume the amount pulled from the user equals the amount received by the pool. When a swap is executed, the pool receives only 98% of the expected input, creating a persistent accounting mismatch. This mismatch is exploitable by arbitrage bots to systematically drain the paired asset (e.g., WETH, USDC) from liquidity pools, causing permanent loss for liquidity providers. This is a well-documented attack vector for tax tokens.

`_update()` function, lines 100–116

→ Whitelist all DEX pair and router addresses via `setTaxExempt` before adding liquidity. Clearly document that this token cannot be safely used with standard AMM protocols without exemptions. Consider whether tax-on-swap is actually desired; if not, exempt trading venues by default at deployment.

Confirmed by: [economist](#)

MEDIUM CVSS 5.3 SC02:2026 – Centralization / Privilege Escalation

Centralization Risk: Owner Can Redirect Tax Revenue and Pause Token at Will

Beyond the LayerZero delegate risk, the owner can at any time: (1) redirect all future tax revenue to an arbitrary address via `setTreasuryAddress`, (2) grant any address tax-free status, and (3) pause all token transfers indefinitely. These capabilities, held by a single key with no timelock, represent a significant trust assumption for token holders and liquidity providers.

`setTreasuryAddress()`, `setTaxExempt()`, `pause()`, `unpause()`

→ Implement a `TimelockController` with at least 24-hour delay on `setTreasuryAddress` and `setTaxExempt`. Consider making pause a multisig-only action. Publish the owner address and governance plan publicly.

Confirmed by: [economist](#), [historian](#)

LOW CVSS 2.5 SC06:2026 – Component / Logic Integrity

Missing tokenURI Update Function Despite MetadataUpdated Event Infrastructure

The contract stores `\_tokenURI` and emits `MetadataUpdated` in the constructor, implying metadata is intended to be updatable. However, no setter function exists to update the URI post-deployment, making the event misleading and the metadata permanently frozen.

`_tokenURI` state variable; `MetadataUpdated` event; constructor

→ Add: `function setTokenURI(string memory newURI) external onlyOwner { _tokenURI = newURI; emit MetadataUpdated(newURI); }`

Confirmed by: developer

INFO CVSS 0.0 SC05:2026 – Input Validation

Liquidity Burn Balance Validation Is Implicit, Not Explicit

The constructor validates `liquidityToBurn <= totalSupply_` but does not explicitly check `liquidityToBurn <= balanceOf(initialOwner)`. In practice this is safe because the full supply is minted to `initialOwner` immediately before, but the implicit dependency is fragile if the constructor logic is ever refactored.

constructor, liquidity burn block

→ Add: `require(liquidityToBurn <= balanceOf(initialOwner), 'Insufficient balance for burn');` after the mint and before the transfer.

Confirmed by: developer

Generated by Sentinelleai on 2026-05-22T15:59:20.272Z. Audit ID: 164c29e8-3dbb-4d31-b61f-184d7181be93.

This report combines deterministic regex+AST sentinels, optional Slither/Mythril static analysis, and a 6-model adversarial LLM panel consolidated by a Judge agent.